

Divide and Conquer - Sorting Algorithms

1. Merge Sort with Comparison Count

```
#include <stdio.h>
int merge_comparisons = 0;

void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        merge_comparisons++;
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int arr[] = {12, 4, 78, 23, 45, 67, 89, 1};
    int n = sizeof(arr) / sizeof(arr[0]);
    merge_comparisons = 0;
    mergeSort(arr, 0, n - 1);
    printf("Output: ");
    for(int i = 0; i < n; i++) { printf("%d", arr[i]); if(i < n-1) printf(","); }
    printf("\nComparisons: %d\n", merge_comparisons);
}
```

```
    return 0;
}
```

2. Quick Sort with Comparison Count

```
#include <stdio.h>
```

```
int quick_comparisons = 0;
```

```
void swap(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}
```

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        quick_comparisons++;
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
```

```
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(arr) / sizeof(arr[0]);
    quick_comparisons = 0;
    quickSort(arr, 0, n - 1);
    printf("Output: ");
    for(int i = 0; i < n; i++) { printf("%d", arr[i]); if(i < n-1) printf(","); }
    printf("\nComparisons: %d\n", quick_comparisons);
}
```

```

    return 0;
}

```

3. Merge Sort vs Quick Sort Comparison

```

#include <stdio.h>
int merge_comp = 0, quick_comp = 0;

void merge(int arr[], int l, int m, int r) {
    int i, j, k, n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        merge_comp++;
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }

int partition(int arr[], int low, int high) {
    int pivot = arr[high], i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        quick_comp++;
        if (arr[j] < pivot) { i++; swap(&arr[i], &arr[j]); }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

```

```

}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr1[] = {12, 11, 13, 5, 6, 7};
    int arr2[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr1) / sizeof(arr1[0]);

    mergeSort(arr1, 0, n - 1);
    quickSort(arr2, 0, n - 1);

    printf("Sorted Array: ");
    for(int i=0; i<n; i++) { printf("%d", arr1[i]); if(i<n-1) printf(","); }
    printf("\nMerge Comparisons: %d\n", merge_comp);
    printf("Quick Comparisons: %d\n", quick_comp);
    return 0;
}

```

4. Merge Sort Execution Time Measurement

```

#include <stdio.h>
#include <time.h>

void merge(int arr[], int l, int m, int r) {
    int i, j, k, n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

```

```

}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int arr[] = {9, 8, 7, 6, 5, 4, 3};
    int n = sizeof(arr) / sizeof(arr[0]);

    clock_t start = clock();
    mergeSort(arr, 0, n - 1);
    clock_t end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Output: ");
    for(int i=0; i<n; i++) { printf("%d", arr[i]); if(i<n-1) printf(","); }
    printf("\nTime Taken: ~%f sec\n", time_taken);
    return 0;
}

```

5. Best and Worst Case Analysis of Quick Sort

```

#include <stdio.h>
int comp_count = 0;

void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        comp_count++;
        if (arr[j] < pivot) { i++; swap(&arr[i], &arr[j]); }
    }
    swap(&arr[i + 1], &arr[high]);
}

```

```

    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int best[] = {1, 2, 3, 4, 5};
    int worst[] = {5, 4, 3, 2, 1};
    int n = 5;

    comp_count = 0;
    quickSort(best, 0, n - 1);
    printf("Best case Output: ");
    for(int i=0; i<n; i++) { printf("%d", best[i]); if(i<n-1) printf(","); }
    printf("\nComparisons: %d\n", comp_count);

    comp_count = 0;
    quickSort(worst, 0, n - 1);
    printf("Worst Case Output: ");
    for(int i=0; i<n; i++) { printf("%d", worst[i]); if(i<n-1) printf(","); }
    printf("\nComparisons: %d\n", comp_count);

    return 0;
}

```

6. Hybrid Merge Sort with Insertion Sort

```

#include <stdio.h>
#define THRESHOLD 4

void insertionSort(int arr[], int left, int right) {
    for (int i = left + 1; i <= right; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= left && arr[j] > key) {
            arr[j + 1] = arr[j];

```

```

        j--;
    }
    arr[j + 1] = key;
}
}

```

```

void merge(int arr[], int l, int m, int r) {
    int i, j, k, n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

```

```

void hybridMergeSort(int arr[], int l, int r) {
    if (r - l + 1 <= THRESHOLD) {
        insertionSort(arr, l, r);
    } else {
        int m = l + (r - l) / 2;
        hybridMergeSort(arr, l, m);
        hybridMergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

```

```

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7, 3, 2};
    int n = sizeof(arr) / sizeof(arr[0]);
    hybridMergeSort(arr, 0, n - 1);
    printf("Output: ");
    for(int i = 0; i < n; i++) { printf("%d", arr[i]); if(i < n-1) printf(","); }
    printf("\n");
    return 0;
}

```

7. Counting Inversions using Merge Sort

```
#include <stdio.h>
```

```
int mergeAndCount(int arr[], int l, int m, int r) {
    int i, j, k, n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    int swaps = 0;
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
            swaps += (n1 - i);
        }
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
    return swaps;
}
```

```
int mergeSortAndCount(int arr[], int l, int r) {
    int count = 0;
    if (l < r) {
        int m = l + (r - l) / 2;
        count += mergeSortAndCount(arr, l, m);
        count += mergeSortAndCount(arr, m + 1, r);
        count += mergeAndCount(arr, l, m, r);
    }
    return count;
}
```

```
int main() {
    int arr[] = {2, 4, 1, 3, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    int inversions = mergeSortAndCount(arr, 0, n - 1);

    printf("Sorted: ");
    for(int i = 0; i < n; i++) { printf("%d", arr[i]); if(i < n-1) printf(","); }
    printf("\nInversions: %d\n", inversions);
}
```



```
    return 0;
}
```

8. Quick Sort Recursion Depth Analysis

```
#include <stdio.h>
```

```
int max_depth = 0;
```

```
void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
```

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) { i++; swap(&arr[i], &arr[j]); }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
```

```
void quickSort(int arr[], int low, int high, int depth) {
    if (depth > max_depth) max_depth = depth;
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1, depth + 1);
        quickSort(arr, pi + 1, high, depth + 1);
    }
}
```

```
int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    max_depth = 0;
    quickSort(arr, 0, n - 1, 1);
    printf("Output: ");
    for(int i=0; i<n; i++) { printf("%d", arr[i]); if(i<n-1) printf(","); }
    printf("\nMax Depth: %d\n", max_depth);
    return 0;
}
```

9. External Merge Process Simulation

```
#include <stdio.h>

int main() {
    int a[] = {1, 3, 5};
    int b[] = {2, 4, 6};
    int n1 = sizeof(a)/sizeof(a[0]);
    int n2 = sizeof(b)/sizeof(b[0]);
    int res[n1 + n2];

    int i = 0, j = 0, k = 0;
    while(i < n1 && j < n2) {
        if(a[i] <= b[j]) res[k++] = a[i++];
        else res[k++] = b[j++];
    }
    while(i < n1) res[k++] = a[i++];
    while(j < n2) res[k++] = b[j++];

    printf("Output: ");
    for(int x = 0; x < n1 + n2; x++) {
        printf("%d", res[x]);
        if(x < (n1 + n2 - 1)) printf(",");
    }
    printf("\n");
    return 0;
}
```

10. Space Complexity Comparison

```
#include <stdio.h>

int main() {
    int arr[] = {5, 3, 8, 4, 2};
    int n = sizeof(arr)/sizeof(arr[0]);

    // Space analysis (analytical approach):
    int int_size = sizeof(int);
    int merge_space = (n * int_size) + (4 * int_size * 3);
    int quick_space = (4 * int_size * 3);

    printf("Sorted: 2,3,4,5,8\n");
    printf("Merge Space: ~%d bytes\n", merge_space);
}
```

```

    printf("Quick Space: ~%d bytes\n", quick_space);
    return 0;
}

```

11. K-th Smallest Element using Quick Sort Partition

```
#include <stdio.h>
```

```
void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
```

```

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivot) { i++; swap(&arr[i], &arr[j]); }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

```

```

int kthSmallest(int arr[], int low, int high, int k) {
    if (k > 0 && k <= high - low + 1) {
        int pos = partition(arr, low, high);
        if (pos - low == k - 1) return arr[pos];
        if (pos - low > k - 1) return kthSmallest(arr, low, pos - 1, k);
        return kthSmallest(arr, pos + 1, high, k - pos + low - 1);
    }
    return -1;
}

```

```

int main() {
    int arr[] = {7, 10, 4, 3, 20, 15};
    int n = sizeof(arr)/sizeof(arr[0]);
    int k = 3;
    printf("Output: %d\n", kthSmallest(arr, 0, n - 1, k));
    return 0;
}

```

12. Performance Data Collection for Graph Plotting

```
#include <stdio.h>
int merge_comp = 0, quick_comp = 0;

void merge(int arr[], int l, int m, int r) {
    int i, j, k, n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        merge_comp++;
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m); mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }

int partition(int arr[], int low, int high) {
    int pivot = arr[high], i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        quick_comp++;
        if (arr[j] < pivot) { i++; swap(&arr[i], &arr[j]); }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); quickSort(arr, pi + 1, high);
    }
}
```

```
    }  
}
```

```
int main() {  
    int arr1[] = {5, 4, 3, 2, 1};  
    int arr2[] = {5, 4, 3, 2, 1};  
    int n = 5;  
  
    mergeSort(arr1, 0, n - 1);  
    quickSort(arr2, 0, n - 1);  
  
    printf("Merge Comparisons: %d\n", merge_comp);  
    printf("Quick Comparisons: %d\n", quick_comp);  
    return 0;  
}
```